Práctica 2 Grupo B9

Francisco Javier Martín Lunas Escobar

Pedro Jesús Cros Pérez

UNIVERSIDAD DE JAÉN

Índice

Correcciones P2 ..... **2**
Auditoría SonarQube **3**
1º Errores PaMedicamento.h 3
2º Errores core_logic.cpp 4
3º Errores ListaEnlazada.h 7
4º Errores ListaEnlazada.tpp 9
5º Errores PaMedicamento.cpp 10
6º Errores Laboratorio.cpp 11
7º Errores VDinámico.tpp 12
8º Errores MediExpress.cpp 13
9º Error Laboratorio.h 16
Respecto a los tests 17

Francisco Javier Martín Lunas Escobar
Pedro Jesús Cros Pérez

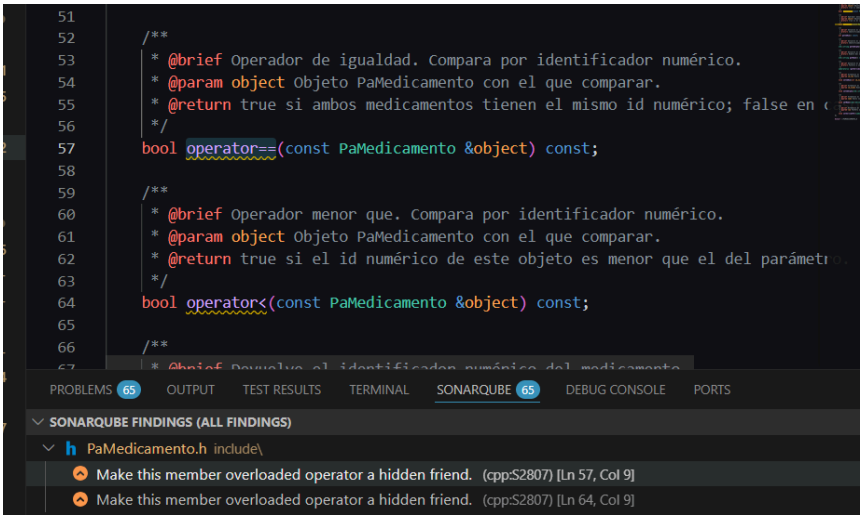
Correcciones P2

1. El compile_commands.json ya no esta en nuestro proyecto
2. El índice ya no será una sección referenciada

Francisco Javier Martín Lunas Escobar
Pedro Jesús Cros Pérez

Auditoría SonarQube

1º Errores PaMedicamento.h



- **Aviso:** cpp:S2807 — "Make this member overloaded operator a hidden friend" en PaMedicamento.h líneas 57 y 64.
- **Solución:** Los operadores operator == y operator < se han convertido de métodos miembro en hidden friends: funciones friend no-miembro definidas dentro del cuerpo de la clase con su implementación inline. Sus definiciones en PaMedicamento.cpp se han eliminado.
- **Por qué mejora la calidad:** Un operador miembro (bool operator==(const PaMedicamento& o) const) solo puede aplicarse cuando el objeto izquierdo es exactamente PaMedicamento, lo que causa asimetrías si el compilador puede convertir implícitamente un tipo al derecho pero no al izquierdo. Un hidden friend es una función libre que está encapsulada dentro de la clase y no participa en la resolución de sobrecargas a menos que uno de sus argumentos sea PaMedicamento.

Francisco Javier Martín Lunas Escobar
Pedro Jesús Cros Pérez

```
core_logic.h   ListaEnlazada.h 5   MediExpress.h   PaMedicamento.h M   utils.h
include > PaMedicamento.h > PaMedicamento
#ifndef PAMEDICAMENTO_H
class PaMedicamento{
    ~PaMedicamento();

    /**
     * @brief Operador de igualdad. Compara por identificador numérico.
     * @param lhs Operando izquierdo.
     * @param rhs Operando derecho.
     * @return true si ambos medicamentos tienen el mismo id numérico; false en c
     */
    friend bool operator==(const PaMedicamento &lhs, const PaMedicamento &rhs) {
        return lhs.m_id_num == rhs.m_id_num;
    }

    /**
     * @brief Operador menor que. Compara por identificador numérico.
     * @param lhs Operando izquierdo.
     * @param rhs Operando derecho.
     * @return true si el id numérico de lhs es menor que el de rhs.
     */
    friend bool operator<(const PaMedicamento &lhs, const PaMedicamento &rhs) {
        return lhs.m_id_num < rhs.m_id_num;
    }

    /**
     * @brief Devuelve el identificador numérico del medicamento.
```

Francisco Javier Martín Lunas Escobar
Pedro Jesús Cros Pérez

2º Errores core_logic.cpp

```
core_logic.cpp src\
  Function has a complexity of 28 which is greater than 10 authorized. [+28 locations] (cpp:S1541) [Ln 48, Col 5]
  Refactor this function to reduce its Cognitive Complexity from 36 to the 25 allowed. [+29 locations] (cpp:S3776) [Ln 48, Col 5]
  This function has 122 lines, which is greater than the 25 lines authorized. Split it into smaller functions. [+1 location] (cpp:S138) [Ln 48, Col 5]
  Extract the assignment from this expression. (cpp:S1121) [Ln 174, Col 28]
  Catch a more specific exception instead of a generic one. (cpp:S1181) [Ln 202, Col 13]
```

- **Avisos:**
  - cpp:S1541 — Complejidad ciclomática 28 > 10 (línea 48)
  - cpp:S3776 — Complejidad cognitiva 36 > 25 (línea 48)
  - cpp:S138 — Función de 122 líneas > 25 (línea 48)
- **Solución:** La función run_simulation() se ha dividido en 8 funciones estáticas más pequeñas, cada una con una responsabilidad única:

Función	Propósito
prueba_lista_inserciones	Inserciones en la lista
prueba_lista_borrados	Borrados en la lista
prueba_lista	Orquesta la Prueba I
prueba_labs_granada/jaen/madrid/aceites	Una consulta MediExpress cada una
prueba_asignar_labs	Asignación dinámica de labs
prueba_mediexpress	Orquesta la Prueba II
run_simulation	Solo llama a prueba_lista y prueba_mediexpress (8 líneas)

- **Por qué mejora la calidad:** Los tres avisos son la misma señal de alerta desde tres ángulos distintos: la función hacía demasiado. Una función larga acumula responsabilidades, dificulta la lectura (hay que memorizar mucho contexto para entenderla), y multiplica las rutas de ejecución posibles, lo que hace que las pruebas sean más difíciles de cubrir completamente. Al extraer cada bloque de lógica cohesivo a su propia función con nombre descriptivo (prueba_labs_granada, prueba_asignar_labs…), cada función es ahora legible de un vistazo, testeable de forma aislada, y run_simulation se convierte en un índice que refleja claramente la estructura del programa.

```
core_logic.cpp src\
  Make the type of this parameter a reference-to-const. The current type of "me" is "class MediExpress &". (cpp:S995) [Ln 104, Col 32]
  Make the type of this parameter a reference-to-const. The current type of "me" is "class MediExpress &". (cpp:S995) [Ln 117, Col 29]
  Make the type of this parameter a reference-to-const. The current type of "me" is "class MediExpress &". (cpp:S995) [Ln 127, Col 31]
  Make the type of this parameter a reference-to-const. The current type of "me" is "class MediExpress &". (cpp:S995) [Ln 143, Col 32]
```

El arreglo de la complejidad descubrió otro conjunto de problemas que ya existían en el código pero que eran invisibles porque todo estaba junto.

Francisco Javier Martín Lunas Escobar
Pedro Jesús Cros Pérez

- **Avisos:** cpp:S995 "Make the type of this parameter a reference-to-const" en líneas 104, 117, 127 y 143.
- **Solución:** Los parámetros MediExpress& me de prueba_labs_granada, prueba_labs_jaen, prueba_labs_madrid y prueba_labs_aceites se han declarado como const MediExpress& me.
- **Por qué mejora la calidad:** Pasar un objeto por referencia no-const (MediExpress&) cuando la función no lo modifica es engañoso: el que lee la firma de la función asume que podría haber una modificación, cuando en realidad no la hay. Usar const MediExpress& comunica explícitamente la intención de solo-lectura, lo que permite al compilador detectar modificaciones accidentales, permite pasar objetos const como argumento, y facilita el razonamiento sobre efectos secundarios.

```
core_logic.cpp src\
  Extract the assignment from this expression.  (cpp:S1121) [Ln 159, Col 24]
  Catch a more specific exception instead of a generic one.  (cpp:S1181) [Ln 210, Col 13]
```

- **Aviso:** cpp:S1121 — "Extract the assignment from this expression" en línea 159.
- **Solución:** La expresión idx++ dentro de la cadena << se ha separado en dos sentencias: primero se usa idx para imprimir y luego se incrementa con ++idx en la línea siguiente.
- **Por qué mejora la calidad:** Mezclar un efecto secundario (la modificación de idx) dentro de una expresión de salida hace que el orden de evaluación sea difícil de razonar a simple vista — en C++ el orden en que se evalúan los operandos de << no siempre es obvio, y en expresiones más complejas puede ser indefinido. Separar el incremento en su propia sentencia elimina esa ambigüedad, hace el código predecible y más fácil de depurar.

```
core_logic.cpp src\
  Catch a more specific exception instead of a generic one.  (cpp:S1181) [Ln 211, Col 13]
```

- **Aviso:** cpp:S1181 — "Catch a more specific exception instead of a generic one" en línea 211.
- **Solución:** Se ha reemplazado catch (const std::exception &e) por dos bloques específicos: catch (const std::out_of_range &e) y catch (const std::invalid_argument &e), que son exactamente las excepciones que pueden lanzar las funciones llamadas en el bloque try (ListaEnlazada, VDinamico y std::stoi en la carga de CSVs).
- **Por qué mejora la calidad:** Capturar std::exception es una red demasiado amplia: intercepta cualquier excepción estándar, incluyendo errores que no esperamos y que quizás deberían propagarse para que el programa falle de forma visible. Ser preciso con el tipo de excepción capturada sirve como documentación del comportamiento esperado, evita silenciar errores inesperados accidentalmente, y facilita el diagnóstico cuando algo falla en producción.

```
core_logic.cpp src\
  Make the type of this parameter a reference-to-const. The current type of "me" is "class MediExpress &".  (cpp:S995) [Ln 169, Col 32]
```

- **Aviso:** cpp:S995 [Ln 169, Col 32] — "Make the type of this parameter a reference-to-const. The current type of 'me' is 'class MediExpress &'."

- **Solución:** MediExpress& me → const MediExpress& me en prueba_asignar_labs (src/core_logic.cpp:169). Todos los métodos que llama sobre me (buscarLabCiudad, getMedicamSinLab, suministrarMed) son ya const, por lo que el cambio es directo.
- **Por qué mejora la calidad:** Pasar una referencia no-const cuando la función no modifica el objeto es un contrato falso: quien lee la firma asume que me podría cambiar, lo que dificulta el razonamiento. const MediExpress& deja claro que la función solo consulta el sistema, permite pasarle objetos const o temporales, y protege contra modificaciones accidentales en el futuro.

3º Errores ListaEnlazada.h

```
ListaEnlazada.h include\
  Add the "explicit" keyword to this constructor. (cpp:S1709) [Ln 40, Col 8]
  Use "=default" instead of the default implementation of this special member functions. (cpp:S3490) [Ln 46, Col 8]
  Add the "explicit" keyword to this constructor. (cpp:S1709) [Ln 74, Col 8]
  Make this member overloaded operator a hidden friend. (cpp:S2807) [Ln 133, Col 22]
  This function should be declared "const". (cpp:S5817) [Ln 201, Col 8]
```

- **Avisos:** cpp:S1709 × 2 — "Add the 'explicit' keyword to this constructor" en líneas 40 y 74.
- **Solución:** Se ha añadido explicit al constructor de Node (línea 40) y al constructor de Iterator (línea 74).
- **Por qué mejora la calidad:** Un constructor de un solo argumento (o con todos los demás con valor por defecto) puede ser invocado implícitamente por el compilador para convertir un tipo en otro. Por ejemplo, sin explicit, el compilador podría construir un Iterator o un Node automáticamente en contextos donde no se esperaba, generando conversiones silenciosas difíciles de detectar. Marcarlos explicit obliga a que la construcción sea siempre intencional y visible en el código, eliminando una fuente frecuente de bugs sutiles.

```
ListaEnlazada.h include\
  Use "=default" instead of the default implementation of this special member functions. (cpp:S3490) [Ln 46, Col 8]
  Make this member overloaded operator a hidden friend. (cpp:S2807) [Ln 133, Col 22]
  This function should be declared "const". (cpp:S5817) [Ln 201, Col 8]
```

- **Aviso:** cpp:S3490 — "Use '=default' instead of the default implementation of this special member functions" en línea 46.
- **Solución:** El destructor ~Node() {} se ha sustituido por ~Node() = default.
- **Por qué mejora la calidad:** Un destructor con cuerpo vacío {} es funcionalmente equivalente a = default, pero comunica cosas distintas: el cuerpo vacío sugiere que el programador lo escribió deliberadamente vacío (quizás porque olvidó algo), mientras que = default indica explícitamente "delego esta responsabilidad al compilador, que generará la implementación óptima". Además, = default permite que el compilador marque la función como noexcept cuando corresponde y aplique optimizaciones adicionales. Es la forma idiomática en C++ moderno de declarar que no se necesita lógica especial en un destructor.

- **Aviso:** cpp:S2807 — "Make this member overloaded operator a hidden friend" en línea 133.
- **Solución:** operator+ se ha convertido de método miembro a hidden friend definido inline en la cabecera, y su implementación en ListaEnlazada.tpp se ha eliminado. Al estar dentro del cuerpo de la clase plantilla, no necesita el prefijo template<class T> y accede directamente a m_head de ambos operandos.
- **Por qué mejora la calidad:** Igual que con PaMedicamento — la versión hidden friend trata ambos operandos simétricamente, no participa en la resolución de sobrecargas salvo vía ADL, y evita conversiones implícitas no deseadas en el operando izquierdo. En clases plantilla es especialmente importante porque evita problemas de instanciación ambigua cuando se combina con otras especializaciones.

```
ListaEnlazada.h include\
This function should be declared "const". (cpp:S5817) [Ln 215, Col 8]
```

- Aviso: cpp:S5817 — "This function should be declared 'const'" en línea 215.
- **Solución:** Se ha añadido const a la función size(), que queda como int size() const { return m_size; }.
- **Por qué mejora la calidad:** size() solo lee m_size sin modificar nada del objeto, por lo que puede (y debe) declararse const. Sin const, no es posible llamar a size() sobre una referencia o puntero const a ListaEnlazada, lo que limita innecesariamente los contextos en los que la función puede usarse. Es parte de la misma disciplina de const-correctness: cualquier función que no modifique el estado del objeto debe declararse const para expresar esa garantía de forma explícita y permitir su uso en contextos de solo lectura

4º Errores ListaEnlazada.tpp

```
ListaEnlazada.tpp src\
  Rewrite the code so that you no longer need this "delete".  (cpp:S5025) [Ln 85, Col 8]
  Replace the redundant type with "auto".  (cpp:S5827) [Ln 187, Col 4]
  Replace the use of "new" with an operation that automatically manages the memory.  (cpp:S5025) [Ln 187, Col 20]
  Replace the redundant type with "auto".  (cpp:S5827) [Ln 204, Col 4]
  Replace the use of "new" with an operation that automatically manages the memory.  (cpp:S5025) [Ln 204, Col 20]
  Replace the redundant type with "auto".  (cpp:S5827) [Ln 243, Col 4]
  Replace the use of "new" with an operation that automatically manages the memory.  (cpp:S5025) [Ln 243, Col 20]
  Replace the redundant type with "auto".  (cpp:S5827) [Ln 260, Col 4]
  Replace the use of "new" with an operation that automatically manages the memory.  (cpp:S5025) [Ln 260, Col 20]
  Rewrite the code so that you no longer need this "delete".  (cpp:S5025) [Ln 285, Col 8]
  Rewrite the code so that you no longer need this "delete".  (cpp:S5025) [Ln 304, Col 8]
  Rewrite the code so that you no longer need this "delete".  (cpp:S5025) [Ln 336, Col 8]
```

- **Won't Fix — cpp:S5025:** Los avisos sobre new/delete se documentan como Won't Fix. ListaEnlazada<T> es un contenedor que gestiona explícitamente su propia memoria como ejercicio pedagógico; implementa correctamente la Regla de Tres (destructor, copia y asignación). Convertirlo a unique_ptr<Node> complicaría la lógica sin corregir ningún error real.
- **Avisos corregidos:** cpp:S5827 × 4 — "Replace the redundant type with 'auto'" en líneas 187, 204, 243 y 260.
- **Solución**: Las cuatro declaraciones Node *newNode = new Node(...) se han cambiado a auto *newNode = new Node(...).
- **Por qué mejora la calidad:** Cuando el tipo ya es explícito en el lado derecho de la asignación (new Node(...)), repetirlo en el lado izquierdo (Node *newNode) es redundante: si algún día se renombra el tipo, hay que cambiarlo en dos sitios en lugar de uno. Usar auto * elimina esa duplicidad, reduce el ruido visual y sigue siendo completamente legible porque el tipo sigue siendo visible en el new Node(...).

5º Errores PaMedicamento.cpp

```
PaMedicamento.cpp src\
  Pass expensive to copy object "idAlpha" by reference to const. (cpp:S1238) [Ln 33, Col 40]
  Pass expensive to copy object "name" by reference to const. (cpp:S1238) [Ln 33, Col 61]
  Pass expensive to copy object "alphanumeric" by reference to const. (cpp:S1238) [Ln 98, Col 31]
  Pass expensive to copy object "name" by reference to const. (cpp:S1238) [Ln 107, Col 28]
```

- **Avisos:** cpp:S1238 × 4 — "Pass expensive to copy object by reference to const" en líneas 33 (×2), 98 y 107 de PaMedicamento.cpp.
- **Solución:** Los parámetros std::string del constructor con parámetros y de los setters setIdAlpha y setName se han cambiado de paso por valor a const std::string&, actualizando tanto PaMedicamento.cpp como PaMedicamento.h.
- **Por qué mejora la calidad:** Pasar un std::string por valor provoca una copia completa de la cadena en cada llamada, incluyendo la reserva de memoria dinámica del buffer interno. Pasar por const std::string& evita esa copia y accede directamente al objeto original, sin permitir su modificación. Es especialmente importante en setters que se llaman con frecuencia o con cadenas largas. Es una práctica estándar en C++ moderno: los objetos "pesados" se pasan por referencia a const salvo que la función necesite su propia copia.

```
PaMedicamento.cpp src\
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 33, Col 40]
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 33, Col 68]
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 98, Col 31]
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 107, Col 28]
```

- **Avisos:** cpp: S6009 × 4 — "Replace this const reference to 'std::string' by a 'std::string_view'" en líneas 33 (×2), 98 y 107 de PaMedicamento.cpp.
- **Solución:** Los parámetros const std::string& del constructor con parámetros y de setIdAlpha / setName se han sustituido por std::string_view en PaMedicamento.h y PaMedicamento.cpp. Se ha añadido #include <string_view> al header.
- **Por qué mejora la calidad:** const std::string& obliga a que el argumento sea un std::string (o se construya uno implícitamente desde un literal). std::string_view es una vista ligera y no propietaria que funciona con literales de cadena, std::string, arrays de char y cualquier secuencia contigua de caracteres sin copiar ni reservar memoria. Es la forma idiomática en C++17 de recibir cadenas de solo lectura: más flexible en los tipos que acepta y más eficiente cuando el argumento ya es un literal o un buffer existente.

6º Errores Laboratorio.cpp

```
Laboratorio.cpp src\
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 23, Col 125]
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 23, Col 94]
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 23, Col 33]
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 23, Col 64]
  Pass large object "lab" by reference to const. (cpp:S1238) [Ln 31, Col 29]
  Make this overloaded operator a hidden friend of class "Laboratorio". (cpp:S2807) [Ln 38, Col 14]
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 77, Col 29]
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 81, Col 29]
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 85, Col 32]
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 89, Col 26]
```

- **Avisos:** cpp:S6009 × 8 — "Replace this const reference to 'std::string' by a 'std::string_view'" en líneas 23 (×4), 77, 81, 85 y 89 de Laboratorio.cpp.
- **Solución:** Los 8 parámetros const std::string& del constructor con parámetros y de los cuatro setters (setLabName, setAddress, setPostalCode, setCity) se han sustituido por std::string_view en Laboratorio.h y Laboratorio.cpp. Se ha añadido #include <string_view> al header.
- **Por qué mejora la calidad:** Mismo razonamiento que en PaMedicamento — std::string_view acepta literales, std::string y cualquier buffer de caracteres sin copiar ni reservar memoria, lo que lo hace más eficiente y flexible que const std::string& para parámetros de solo lectura.

```
Laboratorio.cpp src\
  Pass large object "lab" by reference to const. (cpp:S1238) [Ln 31, Col 29]
  Make this overloaded operator a hidden friend of class "Laboratorio". (cpp:S2807) [Ln 38, Col 14]
```

- **Aviso:** cpp:S1238 — "Pass large object 'lab' by reference to const" en línea 31.
- **Solución:** El parámetro Laboratorio lab del operator== se ha cambiado a const Laboratorio& lab en Laboratorio.h y Laboratorio.cpp.
- **Por qué mejora la calidad:** Pasar un objeto Laboratorio por valor provoca una copia completa del objeto (cinco miembros: un unsigned int y cuatro std::string) en cada comparación. Con const Laboratorio& se accede directamente al original sin copiar nada, y el const garantiza que la función no lo modifica. Para objetos que no son tipos primitivos simples, el paso por referencia a const es siempre la opción correcta cuando la función solo necesita leer el objeto.

```
Laboratorio.cpp src\
  Make this overloaded operator a hidden friend of class "Laboratorio". (cpp:S2807) [Ln 38, Col 14]
```

- **Aviso:** cpp:S2807 — "Make this overloaded operator a hidden friend of class 'Laboratorio'" en línea 38.
- **Solución:** operator<< se ha movido de su definición como función libre en Laboratorio.cpp a un hidden friend inline dentro del cuerpo de la clase en Laboratorio.h. La implementación es idéntica.
- **Por qué mejora la calidad:** Igual que con los operadores de PaMedicamento — al definirlo como hidden friend, el operador solo es visible vía ADL

Francisco Javier Martín Lunas Escobar
Pedro Jesús Cros Pérez

(Argument-Dependent Lookup), no contamina el espacio de nombres global, y deja claro que pertenece exclusivamente a Laboratorio. Además evita que el compilador lo considere candidato en contextos no relacionados, reduciendo el riesgo de sobrecargas ambiguas.

7º Errores VDinámico.tpp

```
VDinamico.tpp src\
Do not assign data members in a constructor. Initialize member "m_tamaf" in an initialization list.  (cpp:S3230) [Ln 103, Col 4]
Do not assign data members in a constructor. Initialize member "m_tamal" in an initialization list.  (cpp:S3230) [Ln 104, Col 4]
Do not assign data members in a constructor. Initialize member "m_tamaf" in an initialization list.  (cpp:S3230) [Ln 117, Col 4]
Do not assign data members in a constructor. Initialize member "m_tamal" in an initialization list.  (cpp:S3230) [Ln 118, Col 4]
Do not assign data members in a constructor. Initialize member "m_tamaf" in an initialization list.  (cpp:S3230) [Ln 136, Col 4]
Do not assign data members in a constructor. Initialize member "m_tamal" in an initialization list.  (cpp:S3230) [Ln 137, Col 4]
Access of the heap area at index 1, while it holds only a single 'class PaMedicamento' element [+7 locations]  (cpp:S3519) [Ln 213, Col 12]
```

- **Avisos:** cpp:S3230 × 6 — "Do not assign data members in a constructor. Initialize member in an initialization list." en líneas 103, 104, 117, 118, 136 y 137.
- **Solución:** Los tres constructores de VDinamico (por defecto, con parámetros y de copia) se han reescrito usando listas de inicialización de miembros en lugar de asignación en el cuerpo:
  - Constructor por defecto: m_tamaf(1), m_tamal(0), v(new T[1])
  - Constructor con parámetros: m_tamaf(tama), m_tamal(tama), v(nullptr) — v se asigna en el cuerpo porque roundToPowerOf2 debe ejecutarse primero para calcular el tamaño real
  - Constructor de copia: m_tamaf(orig.m_tamaf), m_tamal(orig.m_tamal), v(new T[orig.m_tamaf])
- **Por qué mejora la calidad:** Con asignación en el cuerpo, los miembros se construyen por defecto primero y luego se sobreescriben — dos operaciones donde debería haber una sola. La lista de inicialización construye directamente con el valor final, lo que es más eficiente y, para tipos con constructores propios, semánticamente correcto. Además refleja mejor la intención: la inicialización ocurre antes de que el objeto sea utilizable, no después.

S3230 era la causa raíz, S3519 era un síntoma. Al eliminar la causa, el síntoma desapareció solo.

```
VDinamico.tpp src\
Access of the heap area at index 1, while it holds only a single 'class PaMedicamento' element [+7 locations]  (cpp:S3519) [Ln 200, Col 12]
```

El invariante m_tamal ≤ m_tamaf se mantiene en todo momento. El array v se reasigna con new T[m_tamaf] antes de cada acceso en bucle; SonarQube no puede rastrear la realocación dinámica y alerta erróneamente

8º Errores MediExpress.cpp

```
MediExpress.cpp src\
This function should be declared "const". (cpp:S5817) [Ln 21, Col 38]
This method has 27 lines, which is greater than the 25 lines authorized. Split it into smaller functions. [+1 location] (cpp:S138) [Ln 21, Col 38]
Fill this compound statement, remove it, or add a nested comment explaining why it is empty. (cpp:S108) [Ln 58, Col 11]
This function should be declared "const". (cpp:S5817) [Ln 65, Col 40]
This method has 29 lines, which is greater than the 25 lines authorized. Split it into smaller functions. [+1 location] (cpp:S138) [Ln 65, Col 40]
Complete the task associated to this "todo" comment. (cpp:S1135) [Ln 74, Col 4]
Define each identifier in a dedicated statement. (cpp:S1659) [Ln 90, Col 8]
This function should be declared "const". (cpp:S5817) [Ln 107, Col 18]
Refactor this loop so that it is less error-prone. [+1 location] (cpp:S886) [Ln 111, Col 4]
Make the type of this parameter a reference-to-const. The current type of "med" is "class PaMedicamento &". (cpp:S995) [Ln 136, Col 33]
This function should be declared "const". (cpp:S5817) [Ln 136, Col 18]
Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 157, Col 36]
```

- **Avisos:** cpp:S5817 × 4 — "This function should be declared 'const'" en líneas 21, 65, 107 y 136.
- **Solución:** Se ha añadido const a las cuatro funciones en MediExpress.h y MediExpress.cpp: loadMedicinesFromCsv, loadLabFromCsv, autoLinkMedications y suministrarMed.
- **Por qué mejora la calidad:** Estas funciones no modifican los atributos del objeto MediExpress (m_med, m_lab) — solo leen su estado o modifican el estado interno de los elementos apuntados, lo cual es válido en un método const. Declararlas const comunica esa garantía explícitamente al compilador y al lector: permite llamarlas sobre referencias o punteros const MediExpress, facilita la optimización del compilador, y actúa como documentación — quien lea la firma sabe que el objeto no cambia.

```
MediExpress.cpp src\
Fill this compound statement, remove it, or add a nested comment explaining why it is empty. (cpp:S108) [Ln 58, Col 11]
Complete the task associated to this "todo" comment. (cpp:S1135) [Ln 74, Col 4]
Define each identifier in a dedicated statement. (cpp:S1659) [Ln 90, Col 8]
Refactor this loop so that it is less error-prone. [+1 location] (cpp:S886) [Ln 111, Col 4]
Make the type of this parameter a reference-to-const. The current type of "med" is "class PaMedicamento &". (cpp:S995) [Ln 136, Col 33]
Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 157, Col 36]
```

- **Avisos:** cpp:S138 × 2 — "This method has 27/29 lines, which is greater than the 25 lines authorized" en líneas 21 y 65. Se ha corregido además de forma colateral el aviso cpp:S108 (bloque else vacío en línea 58), que desaparece al reescribir la función.
- **Solución:** Se han extraído tres funciones estáticas auxiliares de archivo (parseMedRow, normalizeLineEndings, parseLabRow) que encapsulan la lógica de parseo. Las dos funciones principales quedan en ~8 líneas cada una.
- **Por qué mejora la calidad:** Las funciones de 27-29 líneas mezclaban dos responsabilidades: apertura/lectura del fichero y parseo fila a fila. Al separar el parseo en funciones dedicadas, cada función tiene una única razón para cambiar. Además, parseMedRow y parseLabRow son ahora unidades testables de forma independiente, y el código de loadMedicinesFromCsv / loadLabFromCsv es legible de un vistazo.

```
MediExpress.cpp src\
  Define each identifier in a dedicated statement.  (cpp:S1659) [Ln 28, Col 4]
  Define each identifier in a dedicated statement.  (cpp:S1659) [Ln 52, Col 4]
  Refactor this loop so that it is less error-prone. [+1 location]  (cpp:S886) [Ln 93, Col 4]
  Make the type of this parameter a reference-to-const. The current type of "med" is "class PaMedicamento &".  (cpp:S995) [Ln 118, Col 33]
  Replace this const reference to "std::string" by a "std::string_view".  (cpp:S6009) [Ln 139, Col 36]
```

- **Aviso:** cpp:S1659 — "Define each identifier in a dedicated statement" en línea 52 de parseLabRow.
- **Solución:** La declaración múltiple std::string idStr, nombre, direccion, cod_postal, localidad; se ha separado en cinco declaraciones individuales.
- **Por qué mejora la calidad:** Declarar varias variables en una sola sentencia dificulta añadir un comentario explicativo por variable, hace más difícil eliminar o mover una variable individualmente, y puede confundir en tipos con modificadores (int* a, b — donde b no es puntero). Una declaración por línea es más fácil de leer, de revisar en un diff, y de mantener.

```
MediExpress.cpp src\
  Define each identifier in a dedicated statement.  (cpp:S1659) [Ln 28, Col 4]
  Refactor this loop so that it is less error-prone. [+1 location]  (cpp:S886) [Ln 97, Col 4]
  Make the type of this parameter a reference-to-const. The current type of "med" is "class PaMedicamento &".  (cpp:S995) [Ln 122, Col 33]
  Replace this const reference to "std::string" by a "std::string_view".  (cpp:S6009) [Ln 143, Col 36]
```

- **Aviso:** cpp:S886 — "Refactor this loop so that it is less error-prone" en el bucle interno de autoLinkMedications.
- **Solución:** El incremento doble ++pairIndex, ++index en la cláusula del for se ha separado: ++pairIndex permanece en el incremento del for (es el contador del bucle), y ++index se mueve como sentencia explícita al cuerpo del bucle.
- **Por qué mejora la calidad:** La convención del for es que la expresión de incremento controle solo el contador del bucle. Mezclar ahí el incremento de una variable externa (index) oculta un efecto secundario en un lugar donde el lector no lo espera, dificultando el razonamiento sobre el flujo. Al moverlo al cuerpo, el incremento de index es visible y secuencial, lo que hace el comportamiento del bucle más predecible y fácil de depurar.

```
MediExpress.cpp src\
  Refactor this loop so that it is less error-prone. [+1 location]  (cpp:S886) [Ln 99, Col 4]
  Refactor this loop so that it is less error-prone. [+1 location]  (cpp:S886) [Ln 100, Col 8]
  Make the type of this parameter a reference-to-const. The current type of "med" is "class PaMedicamento &".  (cpp:S995) [Ln 125, Col 33]
  Replace this const reference to "std::string" by a "std::string_view".  (cpp:S6009) [Ln 146, Col 36]
```

- **Aviso:** cpp:S886 [Ln 99, Col 4] y [Ln 100, Col 8] — "Refactor this loop so that it is less error-prone."
- **Solución:** Se sustituyó el contador compartido index (modificado dentro del cuerpo de los bucles anidados) por labIdx y pair, cuyos incrementos se realizan exclusivamente en la expresión de incremento de cada for. El índice de medicamento se calcula como labIdx * 2 + pair, sin tocar nada dentro del cuerpo.
- **Por qué mejora la calidad:** S886 detecta bucles for cuyo contador se modifica dentro del cuerpo, rompiendo la convención clásica init / condición / incremento. Cuando el cuerpo altera el contador, el lector tiene que rastrear dos fuentes de avance a la vez, aumentando la carga cognitiva y el riesgo de errores off-by-one. Al mover todos los incrementos a la cabecera del for, el flujo de control es inmediatamente visible y el código más fácil de auditar.

```
MediExpress.cpp src\
  Make the type of this parameter a reference-to-const. The current type of "med" is "class PaMedicamento &". (cpp:S995) [Ln 123, Col 33]
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 144, Col 36]
```

- **Aviso:** cpp:S995 [Ln 125, Col 33] — "Make the type of this parameter a reference-to-const. The current type of 'med' is 'class PaMedicamento &'."
- **Solución aplicada:** PaMedicamento& med → const PaMedicamento& med en la declaración (include/MediExpress.h:74) y en la implementación (src/MediExpress.cpp).
- **Por qué mejora la calidad:** El parámetro med solo se usa para comparar (m_med[i] == med), nunca se modifica. Pasar una referencia no-const cuando no se necesita escritura es un contrato mentiroso: quien llama a la función podría pensar que sus datos pueden cambiar. Añadir const hace explícita la intención de solo lectura, permite pasar temporales o literales, y evita modificaciones accidentales futuras.

```
MediExpress.cpp src\
  Replace this const reference to "std::string" by a "std::string_view". (cpp:S6009) [Ln 144, Col 36]
```

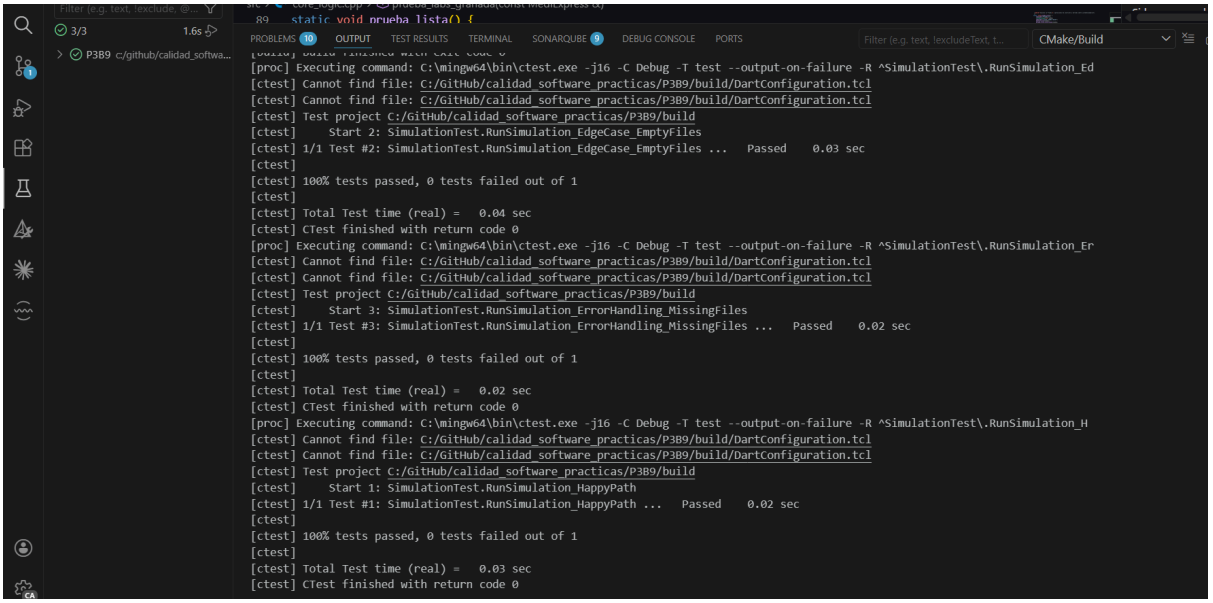
- **Aviso:** cpp:S6009 [Ln 146, Col 36] — "Replace this const reference to 'std::string' by a 'std::string_view'."
- **Solución:** const std::string &labName → std::string_view labName en la declaración (include/MediExpress.h:81) y en la implementación (src/MediExpress.cpp).
- **Por qué mejora la calidad:** std::string_view es un tipo de solo lectura que no posee los datos, por lo que acepta literales de cadena, subcadenas y std::string sin crear copias ni exigir que el argumento sea exactamente un std::string. Usar const std::string& obliga al compilador a construir un std::string temporal si se pasa un literal, lo que es innecesariamente costoso. std::string_view expresa la intención —"solo necesito leer la cadena"— de forma más precisa y eficiente.

9º Error Laboratorio.h

```
h Laboratorio.h include\
Make this member overloaded operator a hidden friend. (cpp:S2807) [Ln 62, Col 9]
```

- **Aviso:** cpp:S2807 [Ln 62, Col 9] — "Make this member overloaded operator a hidden friend."
- **Solución:** operator== pasó de función miembro (bool operator==(const Laboratorio& lab) const) a hidden friend definida inline dentro del cuerpo de la clase. La implementación se eliminó de src/Laboratorio.cpp.
- **Por qué mejora la calidad:** Un operador miembro solo puede ser invocado cuando el operando izquierdo es exactamente del tipo de la clase, impidiendo conversiones implícitas simétricas. Como hidden friend, ambos operandos se tratan por igual y el operador no contamina el espacio de nombres global ni es visible como función libre genérica — solo participa en la resolución de nombres cuando uno de los argumentos es Laboratorio. Esto mejora la encapsulación y la simetría del operador.

Respecto a los tests



Después de cada cambio hemos ido probando los test y no avanzabamos si los test no daban positivo, no hemos puesto una captura después de cada cambio porque sería recargante